



Hello! How is it going?



good

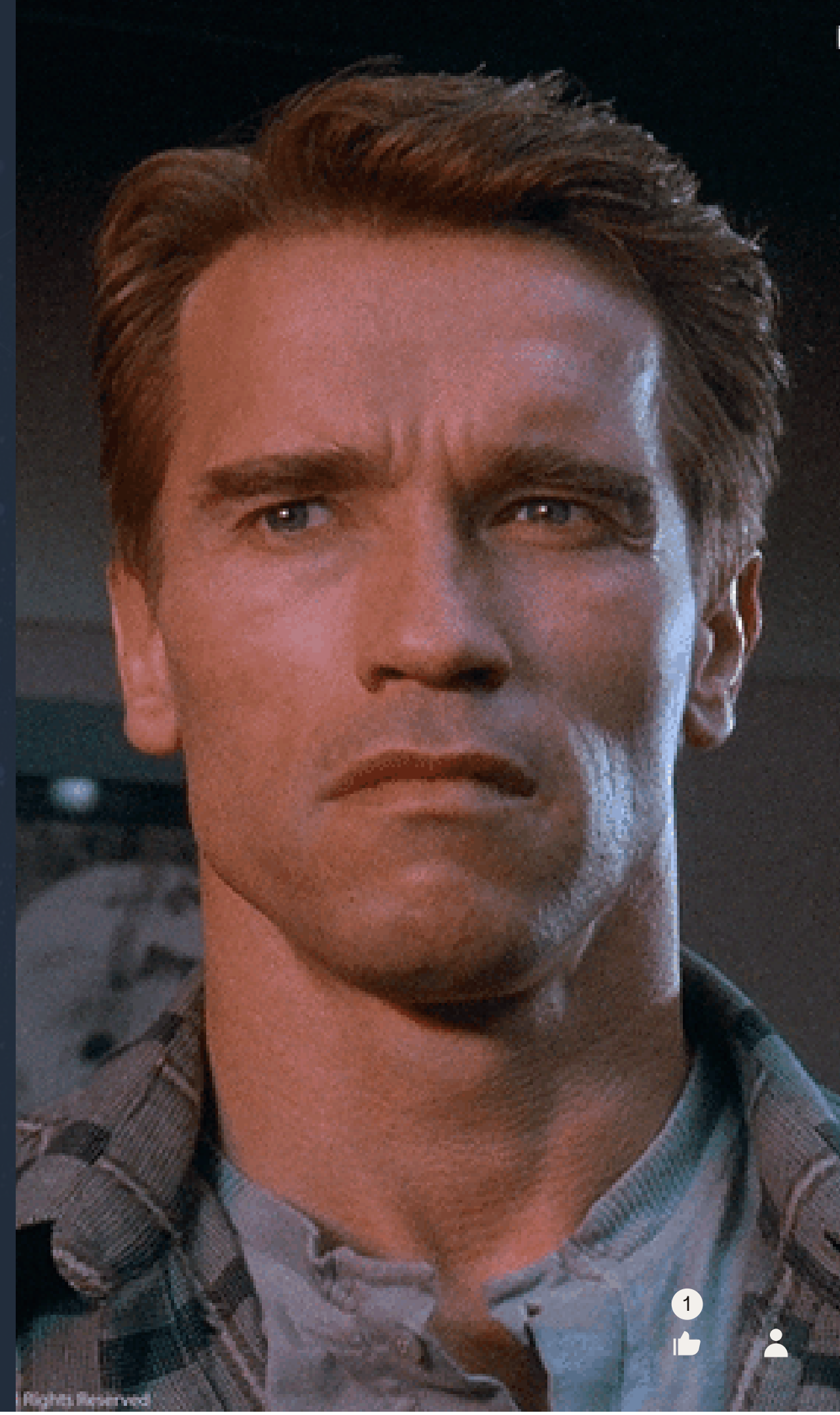
fine

nice

Agenda

- Recall last session
- Discuss current topic
- Practice
- Quiz

Total Recall



What do you remember most from few previous workshops? (short answer)

Semaphore

ExecuterService

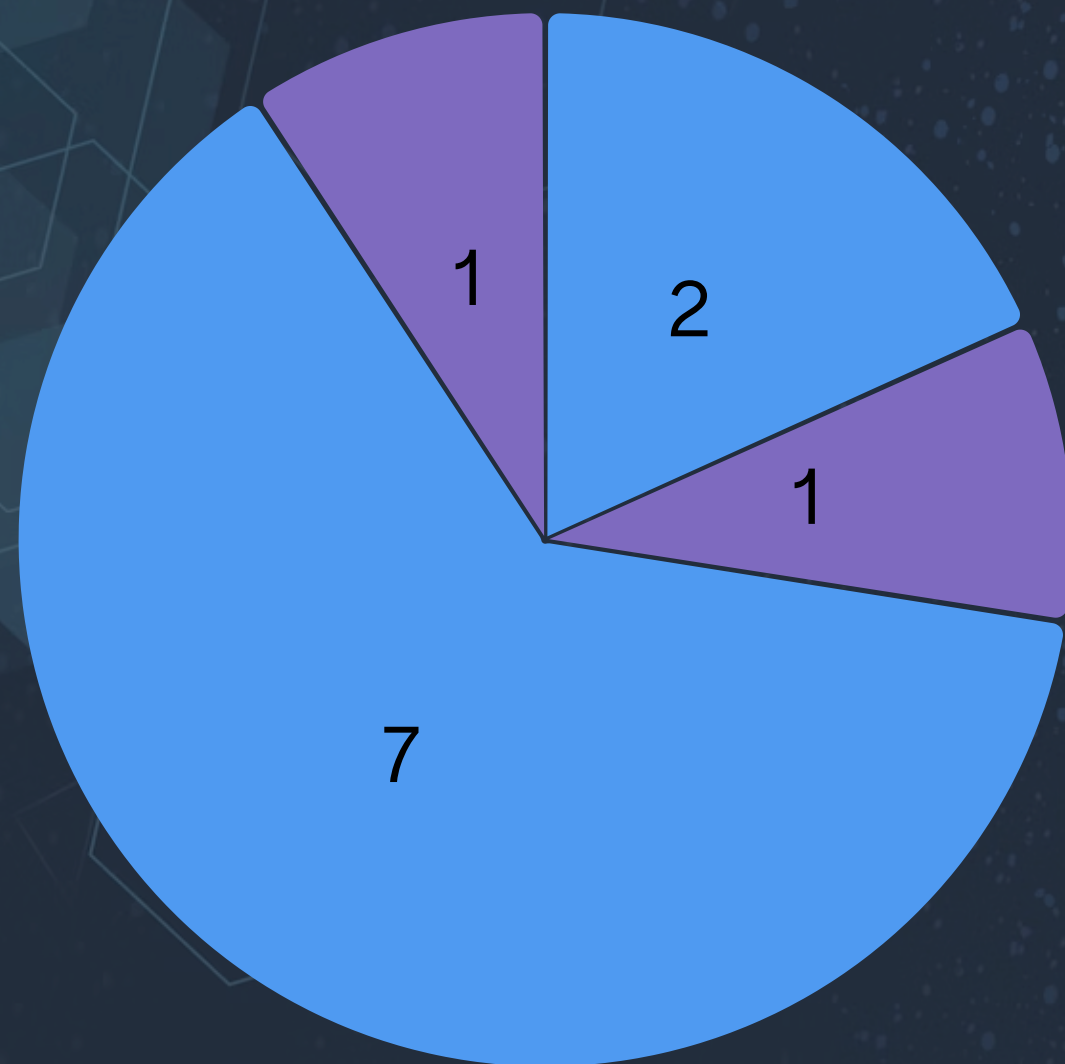
threads

threads

Threads

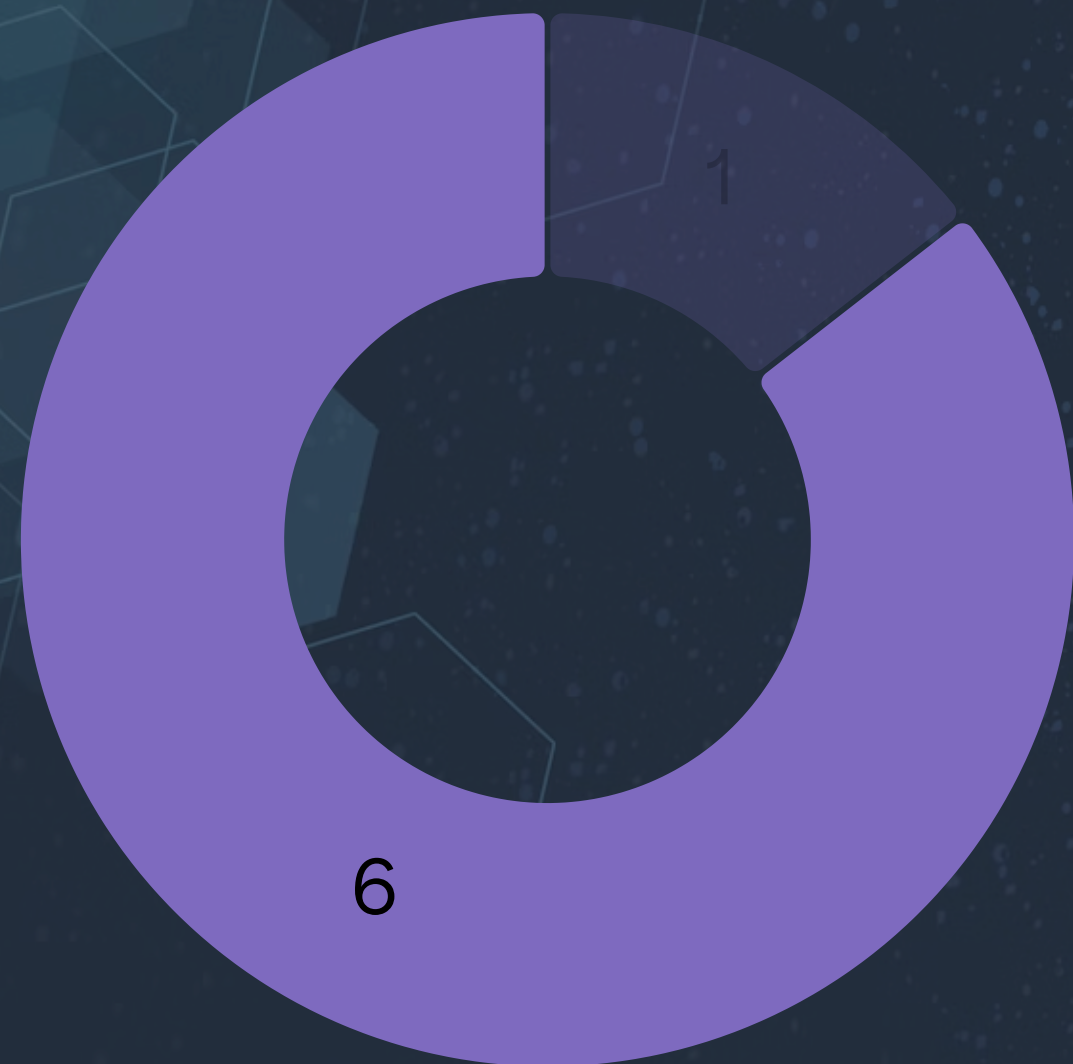
logs

What does semaphore.acquire() do?



- 2 Adds a thread to the queue
- 1 Runs a thread fewer times
- 7 Decreases semaphore permits by 1
- 1 Synchronizes thread flow

Can a daemon thread block JVM termination?



- | | | |
|---|---|---|
| 0 | Yes, always | ✗ |
| 1 | Yes, with a lock | ✗ |
| 0 | No, it pauses | ✗ |
| 6 | No, daemon threads end with main thread | ✓ |

What does CountdownLatch do?



0 Pauses all threads indefinitely 

8 Waits for a number of threads to complete 

0 Returns a thread's priority 

0 Terminates all waiting threads 

What resets a CyclicBarrier?



- | | | |
|---------------------------------------|--|---|
| ☐ 0 | After threads terminate | ✗ |
| ☐ 0 | After Semaphore execution | ✗ |
| ☒ 7 | When the barrier count resets after all threads finish | ✓ |
| ☐ 0 | When threads join a group | ✗ |

Next topic

Random fact

Uzbekistan is one of only two countries in the world that is "doubly landlocked", meaning it is surrounded entirely by other landlocked countries. To reach an ocean or sea, travellers must cross at least two national borders. The only other country with this distinction is Liechtenstein.

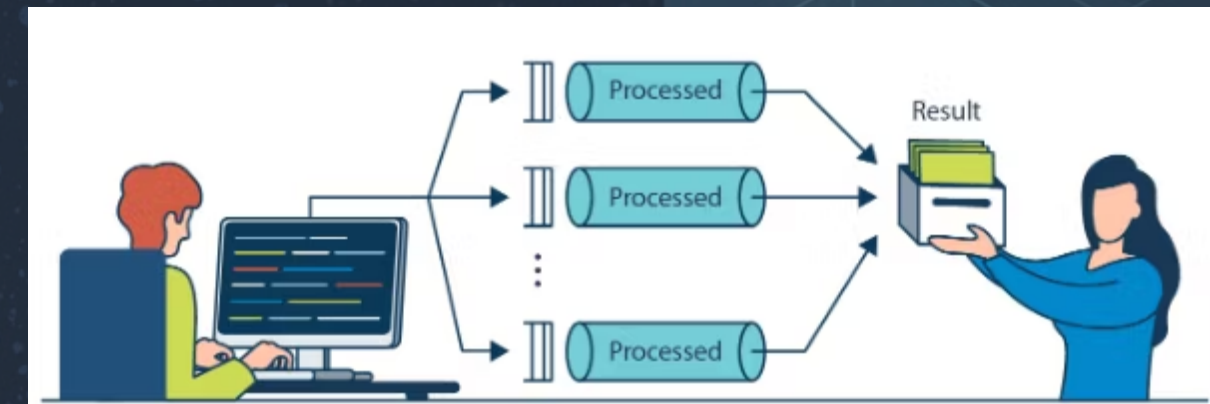


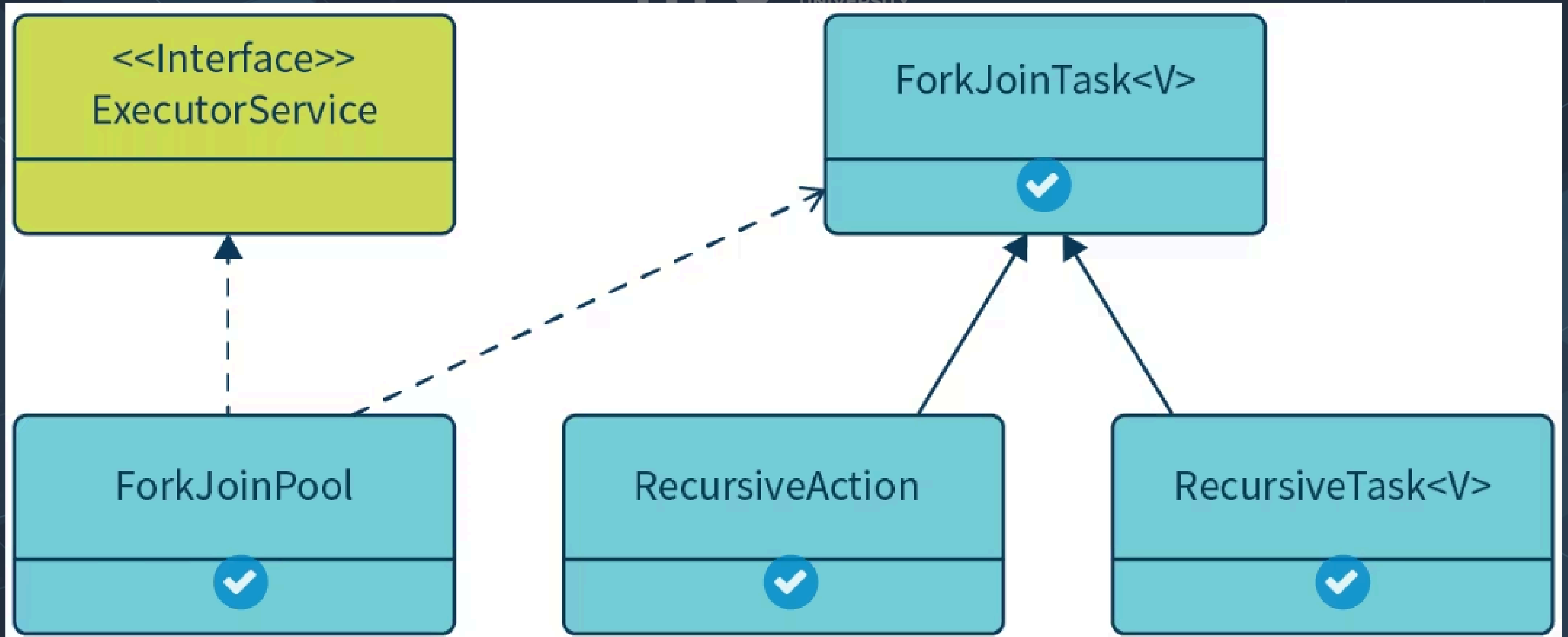
1. Parallel Programming in Java

1.1 Fork/Join framework

In Java 7 (the `java.util.concurrent` package), a new framework was added to support parallel programming: the fork/join framework. This framework:

- Is an implementation of the `ExecutorService` interface
- Allows you to automatically scale the number of available processors under threads
- Makes it easier to create and run threads for execution





Hierarchy

Hierarchy:

- **ExecutorService interface**

- **ForkJoinPool Class**

A pool of threads for managing the execution of tasks of the ForkJoinTask<V> type

- **ForkJoinTask<V> Class**

An abstract class that defines tasks to be executed in a ForkJoinPool thread pool

- **RecursiveAction Class**

An inheritor of the ForkJoinTask<V> class for tasks that do not return values (analogous to a task of the Runnable type)

- **RecursiveTask Class**

An inheritor of the ForkJoinTask<V> class for tasks that return values (analogous to a task of the Callable<V> type)

Signature

- join()

When the join() method is invoked, the thread goes into a waiting state to complete the subtask for which this method is called. This means that the join operation suspends the execution of the current task and waits for the completion of the subtask in order to subsequently combine the result of the subtask with the result of the current task.

- fork()

The fork() method submits a subtask for asynchronous execution. This means that the thread that calls this method continues to execute the current task and the fork operation starts a new parallel subtask.

- invoke()

- invokeAll(...)

These methods implement the divide-and-conquer principle by using recursive algorithm. It allows you to divide tasks into subtasks until subtask becomes simple enough to be solved without further division.

Using the Fork/Join Framework

Creating a Task

To create a task (thread), you must describe a class that extends the following classes:

- `RecursiveAction` if there is no need to return a result
- `RecursiveTask` if you need to return a result

Task Description

The `RecursiveAction` and `RecursiveTask<V>` classes define the abstract method `compute()`, whose body represents the description of the task:

- `Protected abstract void compute()`
- `Protected abstract V compute()`

Completing a Task

To execute tasks, the `ForkJoinPool` class is used. It usually contains the same number of worker threads as the number of available processors. This class also supports the execution of its threads using the **work-stealing** algorithm.

```
if (task_size < threshold_value) {  
1. Solve the task, which is now quite elementary  
} else {  
2. Split the task into two subtasks  
3. Solve each subtask recursively  
4. Combine the results of solving the subtasks  
}
```

Work-stealing algorithm:

- **Initial State**

Each worker thread has a queue of tasks to execute.

- **Current State**

However, there may be a situation where some worker threads can finish their task queue earlier than others.

- **Task Stealing**

Worker threads that have finished their tasks can view other threads' queues and "steal" their tasks.

1.2 Parallelism Based on Streams

Along with the Stream API, Java 8 introduced the concept of parallelism, which uses ForkJoinPool implicitly.

A task can be executed in parallel by calling the `parallel()` method immediately after the stream is created, or by directly creating the stream with the `parallelStream()` method.

Next: explicit and implicit parallelism examples

In Java, a Stream is sequential by default. Think of the ForkJoinPool as a "house" full of workers.

The `.parallel()` call is like giving those workers a command: "Break this blueprint into pieces and work on them together." Without this command, a single worker will take the entire blueprint and work on it alone from start to finish, even if there are seven other colleagues sitting idle in the next room.

An important point: `.parallel()` acts like a "toggle switch." It tells the stream to start creating internal `RecursiveTask` objects and calling those same `fork()` and `join()` methods we discussed earlier.

NB!

Parallel threads running in different parts of the application will compete with each other, so you should limit the size of the pool.



Quiz time

QUIZ

What does the Fork/Join Framework support in Java?

15 ✓



0 ✗

Thread scheduling

0 ✗

Database handling

0 ✗

Data serialization

What does a ForkJoinTask do?

0 ✕

Manages thread lifecycle

1 ✕

Queues thread requests

16 ✓

Splits a task into smaller subtasks

0 ✕

Runs linear processes

What method recursively splits tasks in Fork/Join?



What does join() do in Fork/Join?

0 ✕

Cancels all threads

0 ✕

Executes subtasks synchronously

17 ✓

Waits for task completion and
retrieves result

1 ✕

Releases active threads

Which class is the base for a Fork/Join task?

1 ✗

ThreadExecutor

4 ✗

RecursiveGenerationTask

0 ✗

ParallelManager

12 ✓

RecursiveTask or RecursiveAction

How can streams be executed in parallel?

14 ✓



1 ✗

`stream.forEachOrdered()`

0 ✗

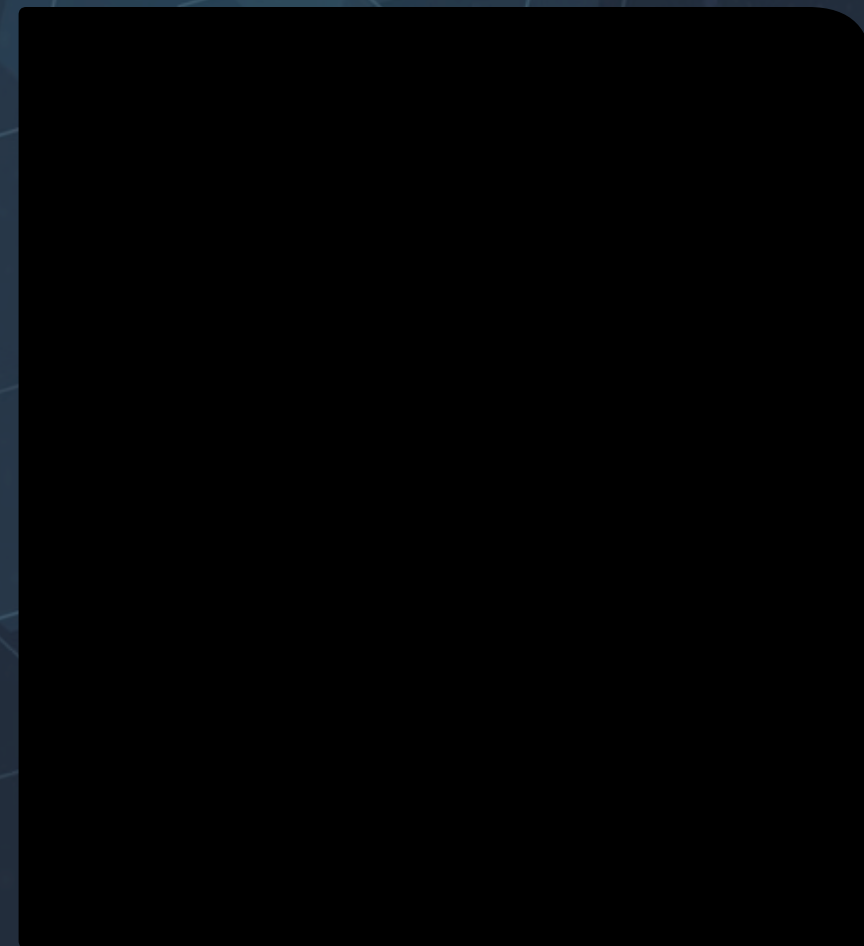
`stream.runPool()`

1 ✗

`stream.sequential()`

What type of parallelism do Java Streams use?

11 ✓



Implicit parallelism

1 ✗

Explicit concurrency

5 ✗



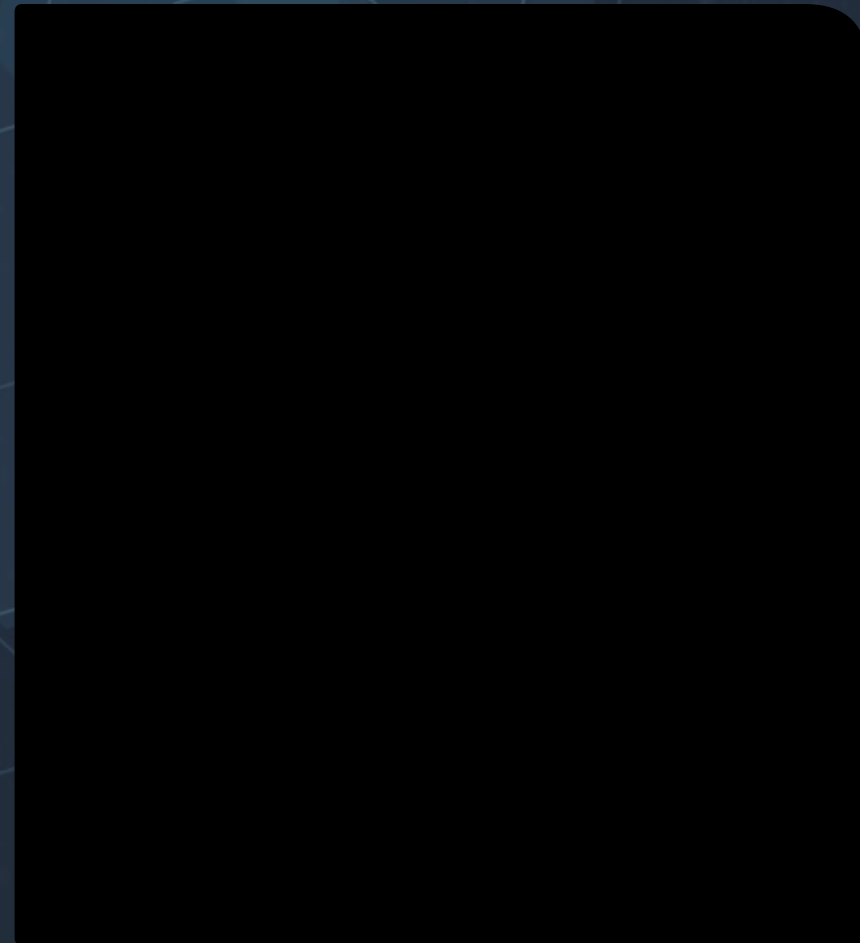
Asynchronous blocking

0 ✗

Thread lifecycle

What is implicit parallelism in Java?

14 ✓



System manages task parallelism

0 ✗

Single-threaded programming

2 ✗

User-created thread controllers

0 ✗

Execution tied to libraries

What does stream.parallel() do?

18 ✓

0 ✗

Starts a thread per operation

0 ✗

Combines blocking threads

Uses Fork/Join for parallel tasks

0 ✗

Executes atomic locks

What does explicit parallelism mean?

1 ✕

Delegating memory to threads

0 ✕

Automated threading

1 ✕

Scoped memory allocation

16 ✓

Threads explicitly created by
developers

Quiz leaderboard



Q&A part

1 question
0 upvotes

